

own signal handler—or just go check the *sigaction* structure in the *man* pages. Now, just receive the signal and ignore it by setting *handler* to *SIG_IGN*. This will not let the child enter a zombie state.

What if the parent finishes before the child (though not the case here, because the server is in an infinite loop)? In that case, the parent can wait for the child using *wait()* or *waitpid()* calls, of which *waitpid* is preferred. These system calls wait for a state change in the child, such as the child being terminated or stopped by a signal, or resumed by a signal (check the *man* pages).

A little science

Now, before moving further, let's talk about TCP here since better code requires a sound understanding of the basics.

In Figure 3, the rectangles represent the states, and the edges, the transitions. We need to visualise the server and client in the diagram. Two edges come out of the CLOSED state; one is *Active Open*. This transition occurs when a client sends a SYN packet to the server, and the system is initiating the connection. Another is *Passive Open*, where the server enters the listen state and waits for connections.

First, from the client side, after the SYN is sent the client enters the *SYN_SENT* state. Now, after receiving the SYN from the server and sending SYN and ACK, it transitions to *ESTABLISHED*. From the *ESTABLISHED* state (where communication takes place), sending a FIN will initiate an *Active Close* to terminate the connection, and enter the *FIN_WAIT_1* state. Receiving ACK will move it to the *FIN_WAIT_2* state. Receiving the FIN from the server will result in sending the ACK, and going to the *TIME_WAIT* state. In

this state, the system waits for twice the MSL (maximum segment lifetime); the recommended value in RFC 1337 is 120 seconds. Linux implements 60 seconds. This state helps reliable termination of connections in case of lost packets, and allows old duplicate segments to expire in the network. Finally, it goes to the *CLOSED* state.

For the server, *passive open* is the *LISTEN* state. Receiving a SYN results in sending SYN and ACK, and going to the *SYN_RCVD* state. Receiving an ACK will take the server to the *ESTABLISHED* state for data communication. Then, receiving a FIN will result in sending an ACK, and will initiate the *passive close* and it going to the *CLOSE_WAIT* state. After the operation completes, the server sends the FIN, and transitions to the *LAST_ACK* state. On receiving the ACK, it will terminate the connection and go to the *CLOSED* state.

Here we can see the 'ThreeHandshake'—the exchange of three packets to establish a TCP connection. It is initiated when the client calls *connect()*. Packet 1 is SYN *x* from client to server; Packet 2 is ACK *x*+1 and SYN *y* from server to client; and Packet 3 is ACK *y*+1 from client to server. Here, *x* is the sequence number from the client, and *y* the sequence number from the server.

To terminate the connection, we need four packets. The client calls *close()* to initiate termination: Packet 1 is FIN *m* from client to server; and Packet 2 is ACK *m*+1 from server to client. Now, the server finishes the operation, and then calls *close()* and sends its FIN *n*. The client sends ACK *n*+1 to terminate the connection.

Here's where I close the connection, even though it was short this time! Next month, I'll be back with a new connection to socket programming... you can now ACK my FIN! 

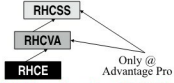
By: Pankaj Tanwar a.k.a. Singularity

The author is a FOSS enthusiast, loves rock music and C, can be reached at pankaj.tux@gmail.com. He is now trying his hand at the cloud—check out cloudrex.in.


redhat®
TRAINING PARTNER
RHCE / RHCVA / RHCSS
Exam Centre



At ADVANTAGE PRO, we do not make tall claims but produce 99% results month after month – TAMIL NADU'S NO. #1 PERFORMING REDHAT PARTNER



Redhat Career Program from **THE EXPERT**

Also get expert training on My SQL-CMDBA, My SQL-CMDEV, PHP, Perl, Python, Ruby, Ajax...

New RHEL 6.0 Exam (RHCSA/RHCE) @ ADVANTAGE PRO on 10th and 24th October'2011

“Do Not Wait! Be a Part of the Winning Team”



ADVANTAGE PRO
 IT TRAINING DIVISION OF
 Vectra Technosoft Pvt. Ltd.
 Open Source Academy

Regd. Off: Wing 1 & 2, IV Floor, Jhaver Plaza, 1A, N.H. Road, Nungambakkam, Chennai - 34.

Ph : 28263529 / 3530 / 3540

Telefax : 28263527

E-mail : enquiry@vectratech.in

www.vectratech.in